Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

# Συστήματα Πολυμέσων και Εικονική Πραγματικότητα

AAC CODEC

Μανωλίδου Ανατολή 10874 amanolid@ece.auth.gr

Χειμερινό Εξάμηνο 2025-2026

# Περιεχόμενα


**1 Εισαγωγή** **3**

**2 Level 1** **4**
2.1 Segmentation Sequence Control (SSC) . . . . . 4
2.1.1 SSC . . . . . 4
2.2 Filter Bank and Inverse Filter Bank . . . . . 5
2.2.1 `filter_bank` . . . . . 6
2.2.2 `i_filter_bank` . . . . . 6
2.3 Encoder and Decoder . . . . . 7
2.3.1 aac_coder_1 . . . . . 7
2.3.2 i_aac_coder_1 . . . . . 7
2.4 Demo . . . . . 8
2.4.1 demo_aac_1 . . . . . 8

**3 Level 2** **9**
3.1 Temporal Noise Shaping (TNS) and Inverse TNS . . . . . 9
3.1.1 `tns` . . . . . 9
3.1.2 `i_tns` . . . . . 10
3.2 Encoder and Decoder . . . . . 10
3.2.1 aac_coder_2 . . . . . 10
3.2.2 i_aac_coder_2 . . . . . 11
3.3 Demo . . . . . 11
3.3.1 demo_aac_2 . . . . . 11

**4 Level 3** **12**
4.1 Psychoacoustic Model . . . . . 12
4.1.1 spreading_function . . . . . 12
4.1.2 psycho . . . . . 12
4.2 Quantizer . . . . . 13
4.2.1 aac_quantizer . . . . . 13
4.2.2 i_aac_quantizer . . . . . 15
4.3 Encoder and Decoder . . . . . 15
4.3.1 aac_coder_3 . . . . . 15
4.3.2 i_aac_coder_3 . . . . . 16
4.4 Demo . . . . . 17
4.4.1 demo_aac_3 . . . . . 17

# 1 Εισαγωγή

Αντικείμενο της παρούσας εργασίας αποτελεί η υλοποίηση ενός κωδικοποιητή/αποκωδικοποιητή ήχου κατά το πρότυπο Advanced Audio Coding (AAC). Πιο συγκεκριμένα, η υλοποίηση στο πλαίσιο της εργασίας αποτελεί μια απλουστευμένη εκδοχή, η οποία βασίστηκε στις παρακάτω βαθμίδες:

1. Sequence Segmentation Control (SSC) και Filter Bank (MDCT), υλοποιημένα στο Level 1
2. TNS, υλοποιημένο στο Level 2
3. Psychoacoustic Model, Quantizer και κωδικοποίηση με Huffman, υλοποιημένα στο Level 3

Τέλος, η κωδικοποίηση έγινε για σήμα ήχου stereo με δειγματοληψία 48000 samples/sec.

# 2 Level 1

Όπως αναφέρθηκε και παραπάνω, σε αυτό το επίπεδο ζητούμενο ήταν η υλοποίηση του Segmentation Sequence Control και του Filter Bank, μαζί με ένα κωδικοποιητή/αποκωδικοποιητή και ένα demo, που να επιδεικνύει τη λειτουργία των βαθμίδων.

## 2.1 Segmentation Sequence Control (SSC)

Σκοπός αυτής της βαθμίδας ήταν η κατηγοριοποίηση των frames ανάλογα με το `frame_type` τους. Συγκεκριμένα, για να αποφύγουμε το "άπλωμα" του θορύβου σε ολόκληρο το frame μας, όταν εντοπίζονται απότομες μεταβολές στην ενέργεια του ηχητικού σήματος (αναφερόμενα και ως attacks) ο μηχανισμός SSC αποφασίζει δυναμικά το μέγεθος του παραθύρου ανάλυσης.

Όταν το ηχητικό σήμα διατηρεί σχετικά αμετάβλητα τα φασματικά του χαρακτηριστικά, η βαθμίδα επιλέγει ένα μακρύ παράθυρο τύπου `ONLY_LONG_SEQUENCE (OLS)`, όπου κρατάμε το original size του frame, δηλαδή 2048 δείγματα. Ωστόσο, σε περιπτώσεις που εμφανίζεται ένας απότομος ήχος, δηλαδή απότομη μεταβολή ενέργειας άρα και ανίχνευση ως attack, η χρήση ενός μεγάλου παραθύρου θα οδηγούσε σε κατανομή του σφάλματος κβαντισμού σε ολόκληρη τη διάρκειά του, κάνοντας τον θόρυβο ακουστό πριν καν ακουστεί ο πραγματικός ήχος.

Για την αντιμετώπιση αυτού του προβλήματος, η βαθμίδα "κοιτάζει μπροστά" στο επόμενο frame μέσω ενός υψιπερατού φίλτρου και ανιχνεύει αν επίκειται attack. Αν ναι, δίνουμε εντολή εναλλαγής σε 8 μικρά παράθυρα τύπου `EIGHT_SHORT_SEQUENCE (ESH)`, δηλαδή 8 subframes των 256 δειγμάτων το καθένα. Η εναλλαγή αυτή περιορίζει τον θόρυβο αυστηρά γύρω από την περιοχή της κρούσης, όπου καλύπτεται από τον ίδιο τον ήχο χάρη στο φαινόμενο της χρονικής κάλυψης.

Τέλος, επειδή δεν είναι εφικτή η απευθείας μετάβαση από ένα συμμετρικό μακρύ παράθυρο σε μικρά, η βαθμίδα SSC χρησιμοποιεί κατάλληλα μεταβατικά, ασύμμετρα παράθυρα (`LONG_START_SEQUENCE (LSS)` και `LONG_STOP_SEQUENCE (LPS)`) για να διασφαλιστεί η σωστή ανακατασκευή του σήματος χωρίς να έχουμε artifacts.

Η συγκεκριμένη βαθμίδα υλοποιείται στο αρχείο `level_1/SSC.py`, το οποίο περιλαμβάνει τη συνάρτηση `SSC`.

### 2.1.1 SSC

**Ορισμός:** `frame_type = SSC(frame_T, next_frame_T, prev_frame_type)`

Όπως είπαμε, ζητούμενο η επιλογή του `frame_type` για το τωρινό μας frame (`frame_T`). Η επιλογή γίνεται ανά κανάλι και βασίζεται στο επόμενο frame (`next_frame_T`), στο οποίο πρώτα πρέπει να του εφαρμόσουμε το παρακάτω υψιπερατό φίλτρο:

$$H(z) = \frac{0.7548 - 0.7548z^{-1}}{1 - 0.5095z^{-1}}$$

Στη συνέχεια, εφαρμόζουμε τη λογική του attack detection όπου χωρίζουμε το επόμενο frame σε 8 τμήματα, υπολογίζουμε την ενέργεια ($s_l^2$) του καθενός και έπειτα το attack value

του ($ds_l^2$), το οποίο μας ορίστηκε ως ο λόγος της ενέργειας ενός τμήματος σε σχέση με το μέσο όρο των προηγουμένων. Σε αυτό το σημείο, στη περίπτωση όπου η μέσος όρος είναι παρα πολύ μικρός και συγκεκριμένα $< 10^{-10}$, θεώρησα πως $ds_l^2 = 0$. Έτσι, σε περίπτωση όπου ικανοποιείται η συνθήκη:

$$s_l^2 > 10^{-3} \quad \text{και} \quad ds_l^2 > 10$$

έστω και για ένα από τα τμήματα, σημειώνουμε πως έχουμε attack.

Έπειτα, με βάση το προηγούμενο frame και το αν εντοπίστηκε attack στο επόμενο frame, κατηγοριοποιούμε το τωρινό frame ως εξής:

- Προηγούμενο `frame_type` (`prev_frame_type`) = `LSS` → Το επόμενο `frame_type` (το τωρινό δηλαδή `frame_T`) αναγκαστικά `ESH`.
- Προηγούμενο `frame_type` = `LPS` → Το τωρινό `frame_type` αναγκαστικά `OLS`.
- Προηγούμενο `frame_type` = `OLS`. Εδώ παίζει ρόλο εάν είχαμε εντοπίσει attack στο επόμενο frame. Οπότε:
  - Εντοπισμός attack → Το τωρινό `frame_type` πρέπει να είναι `LSS`, ώστε να γίνει η μετάβαση.
  - Δεν εντοπίστηκε attack → Το τωρινό `frame_type` πρέπει να είναι `OLS` και πάλι.
- Προηγούμενο `frame_type` = `ESH` → Εδώ και πάλι παίζει ρόλο εάν είχαμε εντοπίσει attack στο επόμενο frame. Οπότε:
  - Εντοπισμός attack → Το τωρινό `frame_type` πρέπει να συνεχίσει να είναι `ESH`.
  - Δεν εντοπίστηκε attack → Το τωρινό `frame_type` πρέπει να είναι `LPS`, ώστε να γίνει η μετάβαση.
- Fallback case: Παρόλο που στην κωδικοποίηση αρχικοποιούμε και το πρώτο frame, για λόγους ασφαλείας προστέθηκε και αυτή η περίπτωση όπου:
  - Εντοπισμός attack → Το τωρινό `frame_type` πρέπει να είναι `LSS` για την μετάβαση.
  - Δεν εντοπίστηκε attack → Το τωρινό `frame_type` πρέπει να είναι `OLS`.

Αφού ολοκληρωθεί η παραπάνω διαδικασία για κάθε ένα κανάλι χωριστά, η τελική απόφαση για το `frame_type` ορίζεται μέσω ενός `decision_table`, το οποίο δημιουργήθηκε με βάση το Table 1 που μας δόθηκε στις οδηγίες.

## 2.2 Filter Bank and Inverse Filter Bank

Η επόμενη βαθμίδα του Level 1 είναι το Filter Bank, μαζί με την αντίστροφη διαδικασία του. Σκοπός αυτής της βαθμίδας είναι η μετάβαση του ηχητικού σήματος από το πεδίο του χρόνου στο πεδίο της συχνότητας, καθώς και η αποσυσχέτιση των δειγμάτων για την αποτελεσματικότερη συμπίεσή τους. Για τον σκοπό αυτό χρησιμοποιήθηκε ο Τροποποιημένος Διακριτός Μετασχηματισμός Συνημιτόνου (Modified Discrete Cosine Transform - MDCT).

Ο μετασχηματισμός MDCT εφαρμόζεται σε επικαλυπτόμενα παράθυρα (κατά 50%), αποφεύγοντας έτσι τα artifacts στα όρια των παραθύρων.

Η συγκεκριμένη βαθμίδα υλοποιείται στο αρχείο `level_1/Filter_Bank.py`, το οποίο περιλαμβάνει τις συναρτήσεις `filter_bank` και `i_filter_bank`.

### 2.2.1 `filter_bank`

**Ορισμός:** `frame_F = filter_bank(frame_T, frame_type, win_type)`

Πριν από την εφαρμογή του μετασχηματισμού, τα δείγματα κάθε παραθύρου πολλαπλασιάζονται στο πεδίο του χρόνου με μία συνάρτηση παραθύρου (διαδικασία windowing). Στην υλοποίησή μας υποστηρίζονται δύο τύποι παραθύρων, όπως ορίζει το πρότυπο AAC: τα παράθυρα Kaiser-Bessel-Derived (`KBD`), τα οποία και χρησιμοποιήθηκαν ως προεπιλογή σε όλη την κωδικοποίηση, και τα ημιτονοειδή (`SIN`).

Η διαδικασία του windowing και του μετασχηματισμού διαφοροποιείται αυστηρά ανάλογα με το `frame_type` που αποφάσισε η βαθμίδα SSC. Διακρίνονται δύο κύριες περιπτώσεις λειτουργίας:

**1. Long Frames (`OLS`, `LSS`, `LPS`):** Αρχικά υπολογίζονται οι συναρτήσεις των long και short παραθύρων βάσει των μαθηματικών τύπων του προτύπου που μας δόθηκαν, ενώ προϋπολογίζεται και ο πίνακας συνημιτόνων για τη βελτιστοποίηση των υπολογισμών των συντελεστών MDCT. Στη συνέχεια, εφαρμόζεται η κατάλληλη συνάρτηση παραθύρωσης:

- `ONLY_LONG_SEQUENCE (OLS)`: Εφαρμόζεται ένα πλήρες, συμμετρικό παράθυρο 2048 σημείων.
- `LONG_START_SEQUENCE (LSS)`: Απαιτεί ένα ειδικό ασύμμετρο παράθυρο. Κατασκευάζεται παραθέτοντας το αριστερό μισό ενός long window, 448 άσσους, το δεξί μισό ενός short window και τέλος 448 μηδενικά. Αυτή η δομή "σβήνει" σταδιακά το σήμα για να επιτρέψει την ομαλή μετάβαση προς τα short windows.
- `LONG_STOP_SEQUENCE (LPS)`: Λειτουργεί ακριβώς αντίστροφα από το LSS, επιτρέποντας την ομαλή επιστροφή από τα short windows σε ένα κανονικό συμμετρικό OLS παράθυρο.

Τέλος, ο μετασχηματισμός MDCT υπολογίζεται ταχύτατα μέσω εσωτερικού γινομένου του πίνακα συνημιτόνων με το παραθυρωμένο σήμα. Από τα 2048 δείγματα εισόδου εξάγονται 1024 συντελεστές συχνότητας.

**2. Short Frames (`ESH`):** Στην περίπτωση αυτών των παραθύρων, αγνοούνται τα 448 δείγματα στην αρχή και στο τέλος του frame, και απομονώνονται τα 1152 κεντρικά δείγματα. Ο κώδικας διατρέχει τα δύο κανάλια και χωρίζει το κεντρικό τμήμα σε 8 μικρά subframes των 256 δειγμάτων το καθένα (με 50% επικάλυψη μεταξύ τους). Στο κάθε subframe εφαρμόζεται το αντίστοιχο short window και υπολογίζεται ο MDCT. Παράγονται 128 συντελεστές ανά subframe, οπότε συνολικά προκύπτουν και εδώ $8 \times 128 = 1024$ συντελεστές.

### 2.2.2 `i_filter_bank`

**Ορισμός:** `frame_T = i_filter_bank(frame_F, frame_type, win_type)`

Αυτή η συνάρτηση εκτελεί την αντίστροφη διαδικασία για την ανακατασκευή του σήματος στο πεδίο του χρόνου. Αρχικά, ανάλογα με τον τύπο του `frame`, εφαρμόζεται ο IMDCT στους συντελεστές εισόδου `frame_F`. Τα ανακατασκευασμένα δείγματα πολλαπλασιάζονται εκ νέου με την ίδια συνάρτηση παραθύρου (`KBD` ή `SIN`).

Ιδιαίτερη διαχείριση χρειαζόταν στα `ESH` frames. Σε αντίθεση με τα long frames που επιστρέφουν απευθείας 2048 δείγματα, στα ESH frames η συνάρτηση εκτελεί ένα Overlap-Add. Τα 8 ανεξάρτητα ανακατασκευασμένα subframes προστίθενται το ένα πάνω στο άλλο με offset 128 δειγμάτων το καθένα. Τα αθροίσματα αυτά τοποθετούνται στο κεντρικό τμήμα ενός ενιαίου πίνακα 2048 δειγμάτων (από τη θέση 448 έως τη θέση 1600), ώστε η έξοδος της συνάρτησης να είναι ομοιόμορφη και έτοιμη για το τελικό εξωτερικό Overlap-Add που θα κάνουμε στον αποκωδικοποιητή.

## 2.3 Encoder and Decoder

Επόμενη βαθμίδα ήταν ο κωδικοποιητής μαζί με τον αποκωδικοποιητή (Encoder/Decoder). Η υλοποίηση περιλαμβάνεται στο αρχείο `level_1/Encoder_Decoder.py`, που περιλαμβάνει τις συναρτήσεις `aac_coder_1` και `i_aac_coder_1`.

### 2.3.1 aac_coder_1

**Ορισμός:** `aac_seq_1 = aac_coder_1(filename_in)`

Όσο αφορά τον Encoder, στην αρχή αφού βεβαιωθούμε πως έχουμε sample rate 48kHz και πως το σήμα ήχου είναι stereo, ορίζουμε ως frame size 2048 δείγματα και hop size 1024 βήματα, ώστε να έχουμε 50% overlap. Έπειτα, χρειάστηκε να εφαρμοστεί padding για τη διαχείριση των άκρων του ηχητικού σήματος. Πιο συγκεκριμένα, έγινε zero-padding στην αρχή του σήματος για τη σωστή εφαρμογή του πρώτου επικαλυπτόμενου παραθύρου. Αντίστοιχα, προστέθηκε padding και στο τέλος του σήματος, ώστε να εξασφαλιστεί αφενός ότι το τελευταίο frame συμπληρώνεται ομαλά, και αφετέρου ότι η βαθμίδα SSC μπορεί να "κοιτάξει" το επόμενο frame χωρίς να προκύψει Out of Bounds error. Επιπλέον, έγινε αρχικοποίηση του πρώτου `frame_type` ως `OLS`, ενώ ως συνάρτηση παραθύρωσης επιλέχθηκε η `KBD`. Στη συνέχεια για κάθε frame, με βάση και τις συναρτήσεις που προαναφέρθηκαν, τα παραγόμενα δεδομένα του κωδικοποιητή οργανώθηκαν στη ζητούμενη δομή λίστας από λεξικά, διαχωρίζοντας τους συντελεστές MDCT για το αριστερό (`chl`) και το δεξί (`chr`) κανάλι.

### 2.3.2 i_aac_coder_1

**Ορισμός:** `x = i_aac_coder_1(aac_seq_1, filename_out)`

Στον Decoder, η διαδικασία ακολουθεί την αντίστροφη πορεία για την ανακατασκευή του αρχικού ηχητικού σήματος. Αρχικά, υπολογίζουμε το συνολικό μήκος του σήματος εξόδου βάσει του πλήθους των κωδικοποιημένων frames και του `hop_size`, και έτσι δεσμεύεται ο απαραίτητος χώρος μνήμης. Στη συνέχεια, γίνεται διάσχιση της λίστας `aac_seq_1` και για κάθε λεξικό ανακτώνται ο τύπος του παραθύρου (`win_type`), ο τύπος του frame

(`frame_type`) και οι συντελεστές MDCT για τα δύο κανάλια. Οι συντελεστές αυτοί ανασυντίθενται στον ενιαίο πίνακα `frame_F` (λαμβάνοντας υπόψη τη διαστασιολόγηση των 8 subframes στην περίπτωση των ESH frames) και τροφοδοτούνται στη συνάρτηση `i_filter_bank`, η οποία επιστρέφει το ηχητικό τμήμα στο πεδίο του χρόνου.

Τέλος, όπως αναφέραμε προηγουμένως έχουμε την εφαρμογή του Overlap-Add. Το κάθε ανακατασκευασμένο frame προστίθεται στον συνολικό πίνακα εξόδου με μια χρονική μετατόπιση ίση με το `hop_size` (1024 δείγματα) σε σχέση με το προηγούμενο. Αυτή η αλληλεπικάλυψη είναι που επιτυγχάνει την τέλεια εξάλειψη της αλλοίωσης που εισήγαγε αρχικά ο MDCT. Τέλος, απορρίπτεται το αρχικό και το τελικό padding που είχαμε προσθέσει κατά την κωδικοποίηση και το πλήρως ανακατασκευασμένο στερεοφωνικό σήμα αποθηκεύεται σε μορφή `.wav`.

## 2.4 Demo

Για την επαλήθευση της ορθής λειτουργίας όλων των παραπάνω βαθμίδων, αναπτύχθηκε η συνάρτηση `demo_aac_1` (στο αρχείο `level_1/demo_level_1.py`). Σκοπός της συγκεκριμένης συνάρτησης, ήταν η εκτέλεση της πλήρους ροής κωδικοποίησης και αποκωδικοποίησης, καθώς και η αξιολόγηση του ανακατασκευασμένου σήματος, μέσω του SNR.

### 2.4.1 demo_aac_1

**Ορισμός:** `SNR = demo_aac_1(filename_in, filename_out)`

Αρχικά, διαβάζεται το αρχικό, ασυμπίεστο αρχείο ήχου (`LicordeCalandraca.wav`) και καλείται ο κωδικοποιητής `aac_coder_1`. Στη συνέχεια, τα παραγόμενα δεδομένα τροφοδοτούνται στον αποκωδικοποιητή `i_aac_coder_1`, ο οποίος ανακατασκευάζει και αποθηκεύει το νέο ηχητικό αρχείο (`Output/output_level1.wav`).

Όπως αναφέρθηκε και παραπάνω, για την αξιολόγηση του συστήματος, υπολογίζεται το SNR. Προκειμένου να γίνει σωστά η σύγκριση, τα δύο σήματα αποκόπτονται στο ελάχιστο κοινό τους μήκος, ώστε να ευθυγραμμιστούν απόλυτα. Στη συνέχεια, υπολογίζεται η συνολική ισχύς του αρχικού σήματος και η ισχύς του θορύβου, το οποίο ορίζεται ως η διαφορά μεταξύ των δύο σημάτων. .

**Αξιολόγηση Αποτελεσμάτων:** Λαμβάνοντας υπ' όψη πως δεν έχει εισαχθεί ακόμη κανενός είδους κβάντιση, το τελικό SNR θα πρέπει να είναι υψηλό και η ισχύς του θορύβου απειροελάχιστη. Παρακάτω φαίνονται τα αποτελέσματα του demo:

```
==================================================
SNR = 253.99 dB
Noise power: 8.07540703653506e-22
==================================================
```

Όντως μπορούμε να δούμε πως το SNR είναι αρκετά υψηλό ενώ η ισχύς του θορύβου αμελητέα.

# 3 Level 2

Σε αυτό το επίπεδο, υλοποιήθηκε η βαθμίδα Temporal Noise Shaping (TNS), καθώς επίσης και ένας κωδικοποιητής/αποκωδικοποιητής και ένα demo που να επιδεικνύει τη λειτουργία των βαθμίδων έως εδώ.

## 3.1 Temporal Noise Shaping (TNS) and Inverse TNS

Ενώ ο μηχανισμός SSC διαχειρίζεται τα μεταβατικά φαινόμενα αλλάζοντας το μέγεθος του παραθύρου, το TNS προσφέρει μια ακόμα πιο λεπτομερή διαχείριση του σφάλματος κβαντισμού . Βασίζεται στη δυαδικότητα χρόνου-συχνότητας, καθώς εφαρμόζοντας τεχνικές Γραμμικής Πρόβλεψης στους συντελεστές MDCT, επιτυγχάνεται η μορφοποίηση του θορύβου κβαντισμού στο πεδίο του χρόνου. Έτσι, ο θόρυβος "κρύβεται" κάτω από τη χρονική περιβάλλουσα του ίδιου του ηχητικού σήματος, αποφεύγοντας περαιτέρω παραμορφώσεις γύρω από απότομους ήχους.

Η συγκεκριμένη βαθμίδα υλοποιείται στο αρχείο `level_2/TNS.py` μέσω των συναρτήσεων `tns` και `i_tns`.

### 3.1.1 `tns`

**Ορισμός:** `frame_F_out, tns_coeffs = tns(frame_F_in, frame_type)`

Ξεκινάμε με μια προεπεξεργασία, ορίζοντας ως `band_table` είτε τον πίνακα `B219a` είτε τον `B219b` ανάλογα με το `frame_type`, καθορίζοντας έτσι το πλήθος των subframes καθώς και το πλήθος των συντελεστών σε κάθε subframe.

Η ανάλυση TNS εφαρμόζεται σε κάθε subframe (1 για `OLS`, `LSS`, `LPS` και 8 για `ESH`) και, με βάση τις οδηγίες του προτύπου, ακολουθεί τα εξής βήματα:

- **Υπολογισμός και Εξομάλυνση Ενέργειας:** Αρχικά, υπολογίζεται η ενέργεια $S_w$ των συντελεστών MDCT για κάθε ψυχοακουστική μπάντα. Επειδή οι ενέργειες αλλάζουν απότομα από μπάντα σε μπάντα, εφαρμόζεται εξομάλυνση δύο περασμάτων, ένα προς τα πίσω και ένα προς τα μπρος.

- **Κανονικοποίηση:** Οι συντελεστές MDCT διαιρούνται με τις εξομαλυμένες τιμές ενέργειας.

- **Γραμμική Πρόβλεψη:** Υπολογίζεται η αυτοσυσχέτιση των κανονικοποιημένων συντελεστών. Στη συνέχεια, για την εύρεση των συντελεστών γραμμικής πρόβλεψης (τάξης $P = 4$), λύνεται το σύστημα των κανονικών εξισώσεων. Σε επίπεδο υλοποίησης, επειδή ο πίνακας αυτοσυσχέτισης είναι συμμετρικός Toeplitz, αξιοποιήθηκε η αποδοτική συνάρτηση `solve_toeplitz` της βιβλιοθήκης `scipy.linalg`.

- **Κβαντισμός και Έλεγχος Ευστάθειας:** Οι συντελεστές LP κβαντίζονται με ανάλυση 4-bit (με βήμα 0.1) και έτσι περιορίζονται στο διάστημα $[-0.8, 0.7]$. Προκειμένου να διασφαλιστεί ότι το αντίστροφο φίλτρο στον αποκωδικοποιητή δεν θα απειριστεί, ελέγχεται η ευστάθειά του υπολογίζοντας τις ρίζες του πολυωνύμου .

Εάν οποιαδήποτε ρίζα βρίσκεται εκτός ή πάνω στον μοναδιαίο κύκλο ($|root| \geq 1.0$), οι συντελεστές θεωρούνται ασταθείς και μηδενίζονται.

- **Εφαρμογή Φίλτρου (FIR):** Οι κβαντισμένοι συντελεστές χρησιμοποιούνται για τη δημιουργία ενός FIR φίλτρου:

$$H_{TNS}(z) = 1 - a_1 z^{-1} - a_2 z^{-2} - \cdots - a_p z^{-p}$$

  το οποίο εφαρμόζεται στους αρχικούς (μη κανονικοποιημένους) συντελεστές MDCT.

Τέλος, η συνάρτηση επιστρέφει τους συντελεστές MDCT μετά την εφαρμογή του TNS (`frame_F_out`), καθώς επίσης και τους κβαντισμένους συντελεστές πρόβλεψης (`tns_coeffs`).

### 3.1.2 `i_tns`

**Ορισμός:** `frame_F_out = i_tns(frame_F_in, frame_type, tns_coeffs)`

Όσον αφορά την αντίστροφη διαδικασία του TNS, σε αυτή τη βαθμίδα παραλαμβάνουμε τους MDCT συντελεστές μετά την εφαρμογή του TNS (`frame_F_in`) και τους κβαντισμένους `tns_coeffs`. Αναλόγως με το `frame_type`, το σήμα χωρίζεται στα κατάλληλα subframes.

Έπειτα, ανά subframe, ανακατασκευάζουμε τους αρχικούς συντελεστές, εφαρμόζοντας το αντίστροφο φίλτρο $H_{TNS}^{-1}(z)$. Το φίλτρο αυτό λειτουργεί πλέον ως IIR και υλοποιείται τροφοδοτώντας τους συντελεστές στον παρονομαστή της συνάρτησης `lfilter`. Η διαδικασία ολοκληρώνεται επιστρέφοντας τους πλήρως ανακατασκευασμένους συντελεστές MDCT στο πεδίο της συχνότητας.

## 3.2 Encoder and Decoder

Επόμενη βαθμίδα ήταν ο κωδικοποιητής μαζί με τον αποκωδικοποιητή. Η υλοποίηση περιλαμβάνεται στο αρχείο `level_2/Encoder_Decoder.py`, που περιλαμβάνει τις συναρτήσεις `aac_coder_2` και `i_aac_coder_2`.

### 3.2.1 aac_coder_2

**Ορισμός:** `aac_seq_2 = aac_coder_2(filename_in)`

Στον Encoder, η αρχική προεργασία του σήματος παραμένει ίδια με το Level 1. Στη συνέχεια, αντί οι συντελεστές MDCT να αποθηκευτούν απευθείας, διαχωρίζονται ανά κανάλι και τροφοδοτούνται στη συνάρτηση `tns`. Η συνάρτηση αυτή επιστρέφει τους τροποποιημένους συντελεστές καθώς και τους υπολογισμένους συντελεστές γραμμικής πρόβλεψης. Έτσι, η τελική δομή του κωδικοποιημένου σήματος (`aac_seq_2`) εμπλουτίζεται: το λεξικό κάθε καναλιού (`chl` και `chr`) πλέον περιέχει το κλειδί `"frame_F"` (MDCT συντελεστές μετά το TNS) και το νέο κλειδί `"tns_coeffs"`.

### 3.2.2 i_aac_coder_2

**Ορισμός:** `x = i_aac_coder_2(aac_seq_2, filename_out)`

Στον Decoder, η διαδικασία αποκωδικοποίησης προσαρμόστηκε αντίστοιχα για να αντιστρέψει το TNS πριν την επιστροφή στο πεδίο του χρόνου. Κατά τη διάσχιση της λίστας `aac_seq_2`, ανακτάμε από κάθε λεξικό τους τροποποιημένους συντελεστές MDCT και τα αντίστοιχα `tns_coeffs`. Αυτά τα δεδομένα εισάγονται πρώτα στη συνάρτηση `i_tns`, η οποία εφαρμόζει το αντίστροφο φίλτρο ξεχωριστά για κάθε κανάλι, ανακατασκευάζοντας τους αρχικούς συντελεστές MDCT.

Από το σημείο αυτό και έπειτα, η ροή ταυτίζεται με εκείνη του Level 1. Οι ανακατασκευασμένοι συντελεστές τροφοδοτούνται στην `i_filter_bank` για τη μετάβαση στο πεδίο του χρόνου και, τέλος, εφαρμόζεται η τεχνική Overlap-Add. Το αρχικό padding απορρίπτεται και το τελικό στερεοφωνικό σήμα αποθηκεύεται σε μορφή `.wav`.

## 3.3 Demo

Όπως και στο Level 1, για την επαλήθευση της ορθής λειτουργίας όλων των παραπάνω βαθμίδων, αναπτύχθηκε η συνάρτηση `demo_aac_2` (στο αρχείο `level_2/demo_level_2.py`). Η συνάρτηση εκτελεί την πλήρη ροή κωδικοποίησης, πλέον και με τον TNS, και αποκωδικοποίησης, υπολογίζοντας στο τέλος την πιστότητα του ανακατασκευασμένου σήματος μέσω του SNR.

### 3.3.1 demo_aac_2

**Ορισμός:** `SNR = demo_aac_2(filename_in, filename_out)`

Αρχικά, διαβάζουμε το αρχικό αρχείο ήχου και καλούμε τον κωδικοποιητή `aac_coder_2`. Στη συνέχεια, τα παραγόμενα δεδομένα τροφοδοτούνται στον αποκωδικοποιητή `i_aac_coder_2`, ο οποίος αντιστρέφει το TNS και το MDCT, και αποθηκεύει το νέο ηχητικό αρχείο (`Output/output_level2.wav`).Ο υπολογισμός του SNR γίνεται ακριβώς όπως και στο Level 1.

**Αξιολόγηση Αποτελεσμάτων:** Η προσθήκη του TNS δεν καθιστά τον κωδικοποιητή απωλεστικό. Παρόλο που οι συντελεστές γραμμικής πρόβλεψης υφίστανται κβαντισμό κατά τον υπολογισμό τους, το FIR φίλτρο ανάλυσης και το IIR φίλτρο σύνθεσης χρησιμοποιούν ακριβώς τους ίδιους κβαντισμένους συντελεστές. Καθώς ελέγχξαμε την ευστάθεια του φίλτρου, η εφαρμογή του $H_{TNS}(z)$ και στη συνέχεια του $H_{TNS}^{-1}(z)$ αποτελεί μια τέλεια αντιστρέψιμη διαδικασία.

Ως αποτέλεσμα, όπως φαίνεται και από την έξοδο του demo παρακάτω, τόσο το παραγόμενο SNR όσο και η ισχύς του θορύβου, έχουν τις ίδιες ακριβώς τιμές που είχαμε και στο Level 1.

```
==================================================
SNR = 253.99 dB
Noise power: 8.0754412421936355e-22
==================================================
```

# 4 Level 3

Σε αυτό το στάδιο εισάγονται το Ψυχοακουστικό Μοντέλο (Psychoacoustic Model) για την εκμετάλλευση των ατελειών της ανθρώπινης ακοής, ο μη-ομοιόμορφος Κβαντιστής για τη συμπίεση των δεδομένων βάσει κατωφλίων ακουστότητας, και τέλος η εντροπική κωδικοποίηση Huffman για την περαιτέρω μείωση του μεγέθους της πληροφορίας.

## 4.1 Psychoacoustic Model

Ο ρόλος του ψυχοακουστικού μοντέλου είναι να υπολογίσει το Signal-to-Mask Ratio - SMR για κάθε ψυχοακουστική μπάντα. Το SMR πρακτικά υποδεικνύει στον κβαντιστή πόσος θόρυβος μπορεί να εισαχθεί σε κάθε μπάντα χωρίς αυτός να γίνει αντιληπτός από το ανθρώπινο αυτί. Η υλοποίηση πραγματοποιήθηκε μέσω των συναρτήσεων `psycho` και `spreading_function`, οι οποίες περιλαμβάνονται στο αρχείο `level_3/demo_Model.py`.

### 4.1.1 spreading_function

**Ορισμός:** `x = spreading_function(i, j, band_table)`

Για την `spreading_function` που αποτελεί μια επιπλέον συνάρτηση από τις ζητούμενενες, αυτή υλοποιήθηκε με βάση τις οδηγίες. Συγκεκριμένα, δέχεται ως όρισμα τα `i` και `j` που είναι τα index της μπάντας που διασπείρει και της μπάντας στην οποία γίνεται η διασπορά, αντίστοιχα, όπως επίσης και το band_tabel, το οποίο μπορεί να είναι είτε ο πίνακας `B219a` είτε ο `B219b`, ώστε μέσα στη συνάρτηση να εξάγουμε τη στήλη bval. Η συνάρτηση, μας επιστρέφει τη τιμή (`x`) της spreding function όπως την ορίσαμε.

### 4.1.2 psycho

**Ορισμός:** `SMR = psycho(frame_T, frame_type, frame_T_prev_1, frame_T_prev_2)`

Σκοπός αυτής της συνάρτησης είναι η υλοποίηση του ψυχοακουστικού μοντέλου για τη μείωση των απαιτούμενων bit κατά την κβάντιση . Το μοντέλο υπολογίζει τον Signal-to-Mask Ratio - SMR, ο οποίος καθορίζει το επιτρεπτό όριο του θορύβου κβάντισης ώστε αυτός να παραμείνει μη αντιληπτός από το ανθρώπινο αυτί.

Αρχικά, ανάλογα με το `frame_type`, επιλέγεται ο αντίστοιχος πίνακας ψυχοακουστικών μπαντών (`B219a` για long frames ή `B219b` για `ESH`) και υπολογίζεται ο πίνακας διασποράς (`spreading_matrix`) μέσω της βοηθητικής συνάρτησης `spreading_function`. Στη συνέχεια, για κάθε `subframe`, ο αλγόριθμος εκτελεί τα βήματα 2-12 από τις οδηγίες:

- **Hann window και FFT:** Το τρέχον frame, καθώς και τα δύο προηγούμενα, πολλαπλασιάζονται με παράθυρο Hann στο πεδίο του χρόνου. Έπειτα, εφαρμόζεται μετασχηματισμός FFT και εξάγονται το μέτρο $r(w)$ και η φάση $f(w)$ για το πρώτο μισό των συχνοτήτων (λόγω συμμετρίας).

- **Predictability:** Χρησιμοποιώντας τα δύο προηγούμενα frames, υπολογίζονται οι προβλέψεις $r_{pred}$ και $f_{pred}$ για το τρέχον frame. Από τη διαφορά της πραγματικής με την προβλεπόμενη τιμή, εξάγεται το μέτρο προβλεψιμότητας

$c(\omega) = \frac{\sqrt{(r(\omega)\cos(f(\omega)) - r_{\text{pred}}(\omega)\cos(f_{\text{pred}}(\omega)))^2 + (r(\omega)\sin(f(\omega)) - r_{\text{pred}}(\omega)\sin(f_{\text{pred}}(\omega)))^2}}{r(\omega) + |r_{\text{pred}}(\omega)|}$

Στη περίπτωση όπου ο παρανομαστής $r(\omega) + |r_{\text{pred}}(\omega)|$ είναι ίσος με το 0 και ο αριθμητής μηδενίζεται οπότε $c(\omega) = 0$

- **Ενέργεια και Τονικότητα ανά Μπάντα:** Οι συχνότητες ομαδοποιούνται στις αντίστοιχες ψυχοακουστικές μπάντες. Υπολογίζεται η ενέργεια $e_b$ και η προβλεψιμότητα $c_b$ της κάθε μπάντας. Τα μεγέθη αυτά συνελίσσονται με τον `spreading_matrix`, προκύπτοντας τελικά ο δείκτης τονικότητας, ο οποίος περιορίζεται ως $tb_b \in (0, 1)$.

- **Υπολογισμός Κατωφλίου:** Βάσει του δείκτη τονικότητας, υπολογίζεται το απαιτούμενο SNR χρησιμοποιώντας τα όρια Tone Masking Noise (18 dB) και Noise Masking Tone (6 dB). Το SNR μετατρέπεται σε κατώφλι ενέργειας $nb_b$.

- **Τελικό SMR:** Το τελικό κατώφλι $npart_b$ προκύπτει επιλέγοντας τη μέγιστη τιμή μεταξύ του κατωφλίου μασκαρίσματος $nb_b$ και του απόλυτου κατωφλίου σε ησυχία $\hat{q}_{thr}$. Τέλος, η συνάρτηση υπολογίζει και επιστρέφει το SMR ως τον λόγο της συνολικής ενέργειας της μπάντας προς το τελικό κατώφλι: $SMR_b = e_b/npart_b$.

Ο πίνακας `SMR` που επιστρέφει η βαθμίδα έχει διαστάσεις (69, 1) για τα long frames και (42, 8) για τα short frames (`ESH`), και αποτελεί την είσοδο για τον μη-ομοιόμορφο κβαντιστή που ακολουθεί.

*Σημείωση:* Ο τελικός υπολογισμός των κατωφλίων κβάντισης $T(b)$ (Βήμα 13 των οδηγιών) δεν ενσωματώθηκε στη συγκεκριμένη συνάρτηση, αλλά εκτελείται απευθείας εντός του κβαντιστή (`aac_quantizer`), λαμβάνοντας ως είσοδο τον πίνακα `SMR` που επιστρέφει το ψυχοακουστικό μοντέλο.

## 4.2 Quantizer

Σκοπός του κβαντιστή, είναι να μετατρέψει τους συντελεστές MDCT σε ακέραια σύμβολα $S$, εισάγοντας θόρυβο κβάντισης ο οποίος ελέγχεται έτσι ώστε να μην ξεπερνά το επιτρεπτό κατώφλι του ψυχοακουστικού μοντέλου. Η διαδικασία αυτή ελέγχεται από έναν scalefactor $a$ για κάθε μπάντα.

Η υλοποίηση της βαθμίδας γίνεται μέσω των συναρτήσεων `aac_quantizer` και `i_aac_quantizer` (στο αρχείο `level_3/Quantization.py`).

### 4.2.1 aac_quantizer

**Ορισμός:** `S, sfc, G = aac_quantizer(frame_F, frame_type, SMR)`

Αναλυτικότερα, όπως και στο `Psychoacoustic_Model`, γίνεται μια προεπεξεργασία αναλόγως το `frame_type` και έπειτα ανα subframe ακουλουθείται η παρακάτω διαδικασία, όπως αυτή μας δόθηκε στις οδηγίες:

- **Υπολογισμός Επιτρεπτού Σφάλματος:** Για κάθε ψυχοακουστική μπάντα $b$, υπολογίζεται η ισχύς του σήματος $P_b$ αθροίζοντας τα τετράγωνα των συντελεστών MDCT. Στη συνέχεια, το κατώφλι του επιτρεπτού σφάλματος υπολογίζεται ως $T_b =$

$P_b/SMR_b$ (σύμφωνα με το Βήμα 13 του Ψυχοακουστικού Μοντέλου, το οποίο υλοποιήθηκε σε αυτή τη βαθμίδα), όπου $SMR$ είναι η τιμή που παρείχε το μοντέλο. Εάν $SMR = 0$, δηλαδή δεν υπάρχει σήμα στη μπάντα, το κατώφλι μηδενίζεται.

- **Αρχικοποίηση Παράγοντα Κλίμακας ($\hat{a}$):** Για να ξεκινήσει η αναζήτηση του βέλτιστου scalefactor, υπολογίζεται μια αρχική εκτίμηση $\hat{a}$ βάσει της μέγιστης απόλυτης τιμής του MDCT στο subframe, αξιοποιώντας τη σταθερά $MQ = 8191$, σύμφωνα με τον τύπο της θεωρίας:

$$\hat{a} = \left\lfloor \frac{16}{3} \log_2 \left( \frac{\max(|X|)^{3/4}}{MQ} \right) \right\rfloor$$

- **Επαναληπτικός Υπολογισμός:** Για κάθε μπάντα αναζητείται ο βέλτιστος ακέραιος παράγοντας $\alpha_b$ . Ξεκινώντας από την τιμή $\hat{a}$, τα σύμβολα κβαντίζονται μέσω του μη-γραμμικού τύπου:

$$S = \text{sign}(X) \cdot \left\lfloor \left( |X| \cdot 2^{-0.25a} \right)^{3/4} + 0.4054 \right\rfloor$$

  Στη συνέχεια, αποκβαντίζονται άμεσα ($\hat{X}$) για να μετρηθεί το παραγόμενο σφάλμα κβάντισης $P_{e,b} = \sum (X - \hat{X})^2$.

  Για την εύρεση της μέγιστης δυνατής συμπίεσης, εφαρμόστηκε λογική αποθήκευσης της τελευταίας αποδεκτής κατάστασης. Όσο το παραγόμενο σφάλμα $P_{e,b}$ παραμένει μικρότερο από το επιτρεπτό κατώφλι $T_b$, αποθηκεύουμε την τρέχουσα τιμή του $a$ (`best_a`) και τον αντίστοιχο πίνακα συμβόλων (`best_S_band`), και κατόπιν αυξάνουμε το $a$ κατά 1. Εάν στην επόμενη επανάληψη το σφάλμα υπερβεί το $T_b$, η επανάληψη τερματίζεται και το σύστημα επιστρέφει αυτόματα στην τελευταία αποδεκτή αποθηκευμένη κατάσταση.

- **Περιορισμοί και Βελτιστοποίηση:** Η επαναληπτική αύξηση του $a$ υπόκειται σε δύο κρίσιμους περιορισμούς, σε περίπτωση παραβίασης των οποίων εκτελείται άμεσο rollback στην τελευταία αποδεκτή κατάσταση:

  1. Η διαφορά του $\alpha_b$ από τον παράγοντα της προηγούμενης μπάντας $\alpha_{b-1}$ δεν επιτρέπεται να υπερβαίνει το $\pm 60$ (`abs(a - alpha[b-1]) > 60`).
  2. Ελέγχεται εάν όλα τα σύμβολα $S$ της μπάντας έχουν μηδενιστεί (`np.all(S_band == 0)`). Αν ισχύει, η περαιτέρω αύξηση του $a$ είναι άσκοπη, καθώς τα σύμβολα έχουν ήδη λάβει τη μέγιστη συμπίεσή τους (έγιναν μηδέν), οπότε ο βρόχος τερματίζεται πρόωρα.

- **Διαφορική Κωδικοποίηση (DPCM):** Επειδή οι τελικοί παράγοντες $\alpha_b$ παρουσιάζουν ισχυρή συσχέτιση μεταξύ γειτονικών μπαντών, δεν αποθηκεύονται αυτούσιοι. Ο παράγοντας της πρώτης μπάντας $\alpha_0$ αποθηκεύεται ως το Γενικό Κέρδος (`Global Gain - G`). Οι υπόλοιποι παράγοντες κωδικοποιούνται διαφορικά (DPCM) ως $sfc_b = \alpha_b - \alpha_{b-1}$, παράγοντας μικρούς αριθμούς, ιδανικούς για την εντροπική κωδικοποίηση που ακολουθεί.

Έτσι, η συνάρτηση μας επιστρέφει το Global Gain `G`, τα scalefactors $sfc$ και τα σύμβολα κβάντισης των συντελεστών MDCT του τρέχοντος frame $S$.

### 4.2.2 i_aac_quantizer

**Ορισμός:** `frame_F = i_aac_quantizer(S, sfc, G, frame_type)`

Στον αποκωδικοποιητή, το σύστημα παραλαμβάνει τα σύμβολα $S$, τις διαφορές $sfc$ και το (`G`). Αρχικά, εκτελεί αντίστροφη διαφορική κωδικοποίηση (Inverse DPCM), θέτοντας $\alpha_0 = G$ και υπολογίζοντας διαδοχικά $\alpha_b = \alpha_{b-1} + sfc_b$. Τέλος, εφαρμόζει τον αντίστροφο μαθηματικό τύπο σε κάθε μπάντα:

$$\hat{X} = \text{sign}(S) \cdot |S|^{4/3} \cdot 2^{0.25\alpha_b}$$

ανακατασκευάζοντας τους συντελεστές MDCT στο πεδίο της συχνότητας, τους οποίους και επστρέφει `frame_F`.

## 4.3 Encoder and Decoder

Το τελικό στάδιο αποτελεί η κωδικοποίηση Huffman. Η υλοποίηση περιλαμβάνεται στο αρχείο `level_3/Encoder_Decoder.py`, που περιλαμβάνει τις συναρτήσεις `aac_coder_3` και `i_aac_coder_3`.

### 4.3.1 aac_coder_3

**Ορισμός:** `aac_seq_3 = aac_coder_3(filename_in, filename_aac_coded)`

Στον Encoder, η αρχική προεργασία του σήματος παραμένει ίδια με αυτή που ακολουθήσαμε στο Level 1 και Level 2. Στη συνέχεια, για κάθε frame, μέσω της βαθμίδας SSC βρίσκουμε το `frame_type` ενώ μέσω του Filter Bank υπολογίζουμε το `frame_F`. Έπειτα, αναλόγως με το `frame_type` ορίζουμε τον πίνακα `band_table` (ίδια προεπεξεργασία που αναφέρθηκε και στις συναρτήσεις `psycho` και `aac_quantizer`). Για κάθε κανάλι, εφαρμόζουμε τη βαθμίδα TNS πάνω στους συντελεστές που πήραμε από το Filter Bank λαμβάνοντας το νέο `frame_F_tns`, και περνάμε το σήμα εισόδου από το Psychoacoustic Model για να πάρουμε το `SMR`. Μετά, τροφοδοτούμε το `SMR` και το `frame_F` (αφού προηγηθεί η μετατροπή του σε μονοδιάστατο πίνακα μέσω της εντολής `flatten('F')` για να υποστηρίζεται η περίπτωση των `ESH`) στον κβαντιστή, υπολογίζοντας παράλληλα για κάθε μπάντα τα κατώφλια $T(b)$ (όπως ακριβώς προηγήθηκε και στην `aac_quantizer`.

Έπειτα, συνεχίζουμε με την κωδικοποίηση κατά Huffman, η οποία έπρεπε να εφαρμοστεί σε δύο επίπεδα:

- **Κωδικοποίηση Scale Factors**
- **Κωδικοποίηση Συμβόλων MDCT**

*Σημείωση:*Σύμφωνα με τις οδηγίες, η συνάρτηση `encode_huff` που έπρεπε να χρησιμοποιήσουμε οριζόταν ως εξής:

```
huff_sec, huff_codebook = encode_huff(coeff_sec, huff_LUT_list, force_codebook)
```

Το όρισμα `force_codebook` ήταν προαιρετικό και προοριζόταν να χρησιμοποιηθεί μόνο στην περίπτωση της κωδικοποίησης των scale factors. Ωστόσο, παρατηρήθηκε ότι η χρήση του προκαλούσε Out of Bounds Error. Συγκεκριμένα, ενώ μια τιμή διαφοράς μπορεί να ήταν εντός των επιτρεπτών ορίων (π.χ. $< 60$), ήταν ταυτόχρονα μεγαλύτερη του 15, με αποτέλεσμα να μην ενεργοποιείται ο μηχανισμός διαφυγής (escape sequence) ώστε να μπορέσει να κωδικοποιηθεί σωστά. Εξετάζοντας τον δοσμένο κώδικα (`huff_utils.py`), διαπιστώθηκε ότι εάν το όρισμα παραλειφθεί, η συνάρτηση εκτελεί αυτόματα escape για τιμές άνω του 15. Για αυτόν τον λόγο, και στις δύο περιπτώσεις κωδικοποίησης το όρισμα παραλείφθηκε. Ως αποτέλεσμα, ήταν σημαντική μια προσθήκη στα ορίσματα της λίστας, ώστε να αποθηκεύσουμε το codebook το οποίο χρησιμοποιήθηκε. Έτσι, προστέθηκε ο πίνακας `sfc_codebook`, ο οποίος για τη περίπτωση των long frames (επειδή υπάρχει μόνο ένα subframe) μετατρέπεται σε μονοδιάστατο.

Για την κωδικοποίηση των συμβόλων MDCT (`S_k`), η διαδικασία εκτελείται επαναληπτικά για κάθε `subframe` και για κάθε μπάντα. Πιο συγκεκριμένα, απομονώνουμε τα κβαντισμένα σύμβολα που αντιστοιχούν στην εκάστοτε μπάντα (`S_band`) και τα περνάμε από τη συνάρτηση `encode_huff`. Η ακολουθία των bits (`stream_band`) αποθηκεύεται στη λίστα `stream_bits`, ενώ ο αριθμός του λεξικού που χρησιμοποιήθηκε αποθηκεύεται στον πίνακα `codebook_arr`. Όπως και στα scalefactors, στην περίπτωση των long frames, επειδή υπάρχει μόνο ένα subframe, ο πίνακας `codebook_arr` μετατρέπεται σε μονοδιάστατο.

Μόλις ολοκληρωθεί η παραπάνω διαδικασία, όλα τα απαραίτητα δεδομένα του καναλιού (συντελεστές TNS, κατώφλια `T`, Global Gain `G`, τα `stream` των scalefactors αλλά και τα `sfc_codebook` που προσθέσα, καθώς και τα `stream` και `codebook` των MDCT) πακετάρονται στο λεξικό (`dict_frame`).

Πριν προχωρήσει ο βρόχος στο επόμενο `frame`, είναι απαραίτητο να ανανεώσουμε τα δεδομένα μνήμης: έτσι το τρέχον `frame_T` αντιγράφεται στο `frame_T_prev_1`, και το παλιό `frame_T_prev_1` στο `frame_T_prev_2`.

Τέλος, το ολοκληρωμένο λεξικό προστίθεται στη συνολική λίστα `aac_seq_3`. Όταν ολοκληρωθεί η επεξεργασία, η τελική δομή αποθηκεύεται σε μορφή αρχείου `.mat`, όπως ακριβώς ζητήθηκε. Η συνάρτηση, μας επιστρέφει τη τελική λίστα `aac_seq_3`.

### 4.3.2 i_aac_coder_3

**Ορισμός:** `x = i_aac_coder_3(aac_seq_3, filename_out)`

Αντίστοιχα, στον Decoder, η διαδικασία αντιστρέφεται προκειμένου να ανακατασκευαστεί το ηχητικό σήμα από τη συμπιεσμένη δομή `aac_seq_3`.

Αρχικά, υπολογίζεται το συνολικό μήκος του σήματος εξόδου και δεσμεύεται ο κατάλληλος χώρος μνήμης (`audio_reconstructed`). Στη συνέχεια, ο αλγόριθμος διατρέχει τη δομή ανά frame και ανά κανάλι, εκτελώντας τα εξής βήματα:

- **Αποκωδικοποίηση Huffman (Scale Factors):** Χρησιμοποιώντας το αποθηκευμένο λεξικό (`sfc_cb`) και το bitstream (`sfc_stream`), η συνάρτηση `decode_huff` επιστρέφει τους αποκωδικοποιημένους παράγοντες κλίμακας. Επειδή ο αλγόριθμος Huffman κωδικοποιεί τα δεδομένα σε ζεύγη, ο Encoder ενδέχεται να κάνει zero padding με σκοπό να κάνει τον αριθμό των στοιχείων άρτιο (π.χ. τα 69 στοιχεία των Long

frames έγιναν 70). Για την επίλυση αυτού, εφαρμόστηκε trimming, κρατώντας τον αριθμό των στοιχείων που ορίζει το `band_table` (`expected_len`). Στη συνέχεια, ο μονοδιάστατος πίνακας αναδιαμορφώνεται σε 2D (αναλόγως τα subframes) κάνοντας χρήση της διάταξης `order='F'`.

- **Αποκωδικοποίηση Huffman (Σύμβολα MDCT):** Παρόμοια διαδικασία ακολουθείται και για τα σύμβολα MDCT. Ο αλγόριθμος διατρέχει τα subframes και τις μπάντες. Εφόσον το αποθηκευμένο λεξικό δεν είναι μηδενικό (`cb > 0`), αποκωδικοποιεί το αντίστοιχο bitstream της μπάντας. Ξανά, εφαρμόζεται trimming στο `decoded_band` για την αφαίρεση πιθανού padding και τα σύμβολα τοποθετούνται στις σωστές τους θέσεις (`start_idx` έως `end_idx`) στον ενιαίο πίνακα `S_k`.

- **Αντίστροφος Κβαντισμός:** Ο πλήρης πλέον πίνακας συμβόλων `S_k`, τα `sfc` και το γενικό κέρδος `G` τροφοδοτούνται στη συνάρτηση `i_aac_quantizer`. Το αποτέλεσμα είναι οι αποκβαντισμένοι συντελεστές MDCT σε μονοδιάστατη μορφή, οι οποίοι μετατρέπονται ξανά στη σωστή 2D διαστασιολόγηση (`frame_F_tns_2d`).

- **Αντίστροφο TNS και Filter Bank:** Εφαρμόζεται το Inverse TNS μέσω της `i_tns` για να αφαιρεθεί η μορφοποίηση θορύβου και οι τελικοί συντελεστές περνούν από την αντίστροφη συστοιχία φίλτρων (`i_filter_bank`) για τη μετάβαση στο πεδίο του χρόνου.

Τέλος, το σήμα εξόδου του κάθε `frame` προστίθεται στον κεντρικό πίνακα `audio_reconstructed` ακολουθώντας Overlap-Add με χρονική μετατόπιση `hop_size`. Αφού ολοκληρωθεί η επεξεργασία όλων των frames, το σύστημα απορρίπτει το αρχικό zero-padding που είχε εισαχθεί κατά την κωδικοποίηση και αποθηκεύει το τελικό ηχητικό αποτέλεσμα σε μορφή `.wav`.

## 4.4 Demo

Για την τελική αξιολόγηση του πλήρους κωδικοποιητή, αναπτύχθηκε η συνάρτηση `demo_aac_3`. Σε αντίθεση με τα προηγούμενα επίπεδα όπου η διαδικασία ήταν μη απωλεστική, το Level 3 ενσωματώνει τον κβαντιστή, συνεπώς αναμένεται μια ρεαλιστική τιμή SNR, καθώς και σημαντική μείωση του όγκου των δεδομένων.

### 4.4.1 demo_aac_3

**Ορισμός:** `SNR, bitrate, compression = demo_aac_3(filename_in, filename_out, frame_aac_coded))`

Εδώ πλέον εκτελούμε την πλήρη αλυσίδα κωδικοποίησης (αποθηκεύοντας το `encoded_seq.mat`) και αποκωδικοποίησης (παράγοντας το `output_level3.wav`), και στη συνέχεια υπολογίζουμε τρεις βασικές μετρικές:

- **SNR:** Υπολογίζεται ακριβώς όπως και στα προηγούμενα επίπεδα.

- **Ρυθμός Μετάδοσης Δεδομένων (Bitrate):** Για τον υπολογισμό του Bitrate σε bps, προσμετρώνται όλα τα δεδομένα που "στέλνονται" στον αποκωδικοποιητή. Συγκεκριμένα, για κάθε frame και κάθε κανάλι, αθροίζονται:

1. Της επικεφαλίδας για τον τύπο παραθύρου και frame (3 bits αφού έχουμε 4 frame types και 2 window types). Αυτά αθροίζονται μόνο ανά frame.
2. Το πραγματικό μήκος των Huffman bitstreams (για τα `S_k` και τα `sfc`).
3. Τα bits για τους δείκτες των λεξικών (θεωρώντας 4 bits ανά `codebook`, αφού έχουμε 11) και για τα scalecators και για τους MDCT.
4. Το Γενικό Κέρδος `G` (θεωρώντας 8 bits ανά τιμή).
5. Οι κβαντισμένοι συντελεστές TNS (θεωρώντας 4 bits ανά συντελεστή λόγω του 4-bit quantization).

Το συνολικό άθροισμα των bits διαιρείται με τη χρονική διάρκεια του αρχικού ηχητικού αρχείου.

- **Λόγος Συμπίεσης (Compression Ratio):** Υπολογίζεται ως το πηλίκο του αρχικού ασυμπίεστου Bitrate προς το Bitrate του κωδικοποιητή μας. Το αρχικό Bitrate ορίζεται στα 1536000 bps, δεδομένου ότι έχουμε στερεοφωνικό ήχο με συχνότητα δειγματοληψίας 48 kHz και ανάλυση 16 bits ανά δείγμα.

**Αξιολόγηση Αποτελεσμάτων:** Κατά την εκτέλεση του demo, τα αποτελέσματα που προέκυψαν επιβεβαιώνουν την λειτουργία της απωλεστικής συμπίεσης:

```
==================================================
Final SNR         = 10.50 dB
Original Bitrate  = 1536000.00 bps
Encoded Bitrate   = 264637.28 bps
Compression Ratio = 5.80 : 1
==================================================
```

Συγκριτικά με τα Level 1 και 2, το SNR μειώθηκε σημαντικά στα 10.50 dB λόγω της εισαγωγής του κβαντιστή, ωστόσο, η ακρόαση του τελικού αρχείου επιβεβαιώνει πως παρατηρείται μια απειροελάχιστη ακουστική αλλοίωση σε σχέση με το αρχικό σήμα, η οποία σε πρώτη ακρόαση μπορεί να μην είναι καν ιδιαίτερα αντιληπτή.

Παράλληλα, το μέγεθος της πληροφορίας μειώθηκε κατά 5.80 φορές, με τον τελικό ρυθμό μετάδοσης να διαμορφώνεται στα ∼264.6 kbps. Σύμφωνα με τη σχετική βιβλιογραφία της IEEE που μας δόθηκε, ο κωδικοποιητής AAC επιτυγχάνει ποιότητα ραδιοφωνικής εκπομπής (ITU-R broadcast quality) ήδη από τα 128 kbps για στερεοφωνικό σήμα, γεγονός που αποδεικνύει ότι ο ρυθμός των ∼264 kbps που επιτεύχθηκε στο σύστημά μας καλύπτει τις απαιτήσεις για ακουστική διαφάνεια, διατηρώντας ταυτόχρονα έναν καλό λόγο συμπίεσης.